
CS2610: Computer Organization and Architecture

Lab 11 Report Branch Predictor

Name: Ananya Uday Naik
Roll No.: CS24B066
Date: 18th April 2026

1 Objective

The objective of this lab is to handle **control hazards** that arise from conditional branch instructions in a two-stage RISC-V pipeline. Control hazards occur because the processor does not know whether a branch will be taken or not until the branch condition is actually evaluated in Stage 2 (Execute). By that time, Stage 1 has already fetched the next instruction — which may be the wrong one if the branch is taken. This leads to wasted cycles and incorrect execution.

To solve this problem, a **dynamic branch predictor** based on a **2-bit saturating counter** is designed and integrated into the pipeline. The predictor makes an informed guess about whether a branch will be taken or not, and the pipeline proceeds based on that guess. If the prediction turns out to be wrong, the incorrectly fetched instruction is flushed and the pipeline is redirected to the correct path. This lab builds directly on top of the data hazard solution implemented in Lab 10, adding control hazard handling as an orthogonal feature.

2 Pipeline Overview

The processor implements a **two-stage pipeline**:

- **Stage 1 (S1) — Fetch, Decode, Register Read:** In this stage, the instruction is fetched from instruction memory using the current Program Counter (PC). The fetched instruction is then decoded by the instruction decoder, which identifies the instruction type, extracts register addresses (rs1, rs2, rd), immediate values, and control signals. Register values are also read in this stage. Crucially, since decoding is completed early in S1, the processor can identify a branch instruction and compute the branch target address in this same stage without waiting for Stage 2.
- **Stage 2 (S2) — Execute, Memory, Writeback:** In this stage, the ALU performs the actual computation. For branch instructions, the ALU evaluates the branch condition (e.g., whether two registers are equal) and determines the actual outcome — taken or not taken. Memory operations (loads and stores) and register writebacks also happen here.

2.1 Pipelining Change from Previous Lab

In the original (non-pipelined) design, a T flip-flop was present in Stage 1 to control when instructions were processed. To convert this into a proper pipeline, the T flip-flop was **removed** and the `ready_stall` signal output of the decoder was connected **directly** to the PC enable signal. This allows the PC to advance and fetch a new instruction every cycle, enabling true overlap of Stage 1 and Stage 2 execution. Without this change, the pipeline would not function correctly as the fetch stage would stall unnecessarily.

3 Branch Predictor Architecture

3.1 Overview

The branch predictor is implemented as a standalone subcircuit in Logisim. Its purpose is to maintain a table of historical branch behaviour and use that history to predict, at fetch time, whether the current branch instruction will be taken or not. The predictor is queried in Stage 1 when a branch instruction is detected, and it is updated in Stage 2 after the actual branch outcome is known.

The predictor has the following interface signals:

Signal	Direction	Description
<code>branch_sel</code>	Input	Indicates that the current instruction is a branch
<code>prev_pc</code>	Input	PC of the branch instruction from the previous cycle (used for BHT update)
<code>branch_taken</code>	Input	Actual outcome from ALU (1 = taken, 0 = not taken)
<code>clock</code>	Input	Clock signal for the registers
<code>reset</code>	Input	Resets all BHT entries to 00 (Strongly Not Taken)
<code>pred_out</code>	Output	Prediction for the current branch (1 = predict taken, 0 = predict not taken)
<code>curr_pc</code>	Output	Passes through the current PC for internal use

3.2 Branch History Table (BHT)

The core data structure of the predictor is the **Branch History Table (BHT)**. It is an array of 64 entries, where each entry stores a **2-bit saturating counter** representing the recent branch behaviour at that address. The BHT is indexed using bits `PC[7:2]` — the lower 6 bits of the PC (after ignoring the bottom 2 bits which are always zero for aligned instructions). Since $2^6 = 64$, these 6 bits uniquely address all 64 entries.

Parameter	Value
Number of entries	64
Entry width	2 bits (saturating counter)
Index bits	<code>PC[7:2]</code> (bits 7 down to 2 of the PC)
Total BHT size	$64 \times 2 = 128$ bits

In Logisim, each of the 64 entries is implemented as a **2-bit register** (D flip-flop pair) that holds the current counter state. All 64 registers share the same clock and reset lines.

3.3 2-bit Saturating Counter

Each BHT entry is a **2-bit saturating counter** that acts as a finite state machine (FSM) with four states.

State	Binary	Prediction	Meaning
SN	00	Not Taken	Strongly Not Taken
WN	01	Not Taken	Weakly Not Taken
WT	10	Taken	Weakly Taken
ST	11	Taken	Strongly Taken

The prediction is determined by the **Most Significant Bit (MSB)** of the counter:

- MSB = 1 (states 10 or 11) $\Rightarrow$ **Predict Taken**
- MSB = 0 (states 00 or 01) $\Rightarrow$ **Predict Not Taken**

The advantage over a 1-bit predictor is clear: if a branch is usually taken but occasionally not taken (e.g., the loop exit condition), the 2-bit counter will not immediately switch to “predict not taken” after one miss. It requires two consecutive not-taken outcomes before the prediction changes, improving overall accuracy.

3.4 Read Path: MUX-based Prediction Lookup

When an instruction is fetched in Stage 1 and identified as a branch, the predictor is queried immediately. The read path works as follows:

1. The current PC[7:2] (6 bits) is used as the **select input** of a **64-to-1 MUX**.
2. Each of the 64 inputs of the MUX is connected to one of the 64 BHT register outputs (2-bit counter values).
3. The MUX selects and outputs the 2-bit counter corresponding to the current PC.
4. The **MSB of the output** is taken as the prediction:

$$\text{pred_out} = \text{counter}[1]$$

5. If **pred_out** = 1: the pipeline predicts the branch is taken, and the next PC is set to PC + ImmB.
6. If **pred_out** = 0: the pipeline predicts not taken, and the next PC is set to PC + 4.

3.5 Write Path: DEMUX-based BHT Update

After Stage 2 resolves the branch (i.e., the ALU computes the actual outcome), the BHT entry for that branch must be updated. However, because the pipeline is two-staged, the update happens **one cycle after** the instruction was in Stage 1. This is why the update uses **prev_pc** — the PC value from the previous cycle — rather than the current PC. The write path works as follows:

1. `prev_pc[7:2]` (6 bits) is used as the **select input** of a **1-to-64 DEMUX**.
2. The DEMUX routes the update enable signal to exactly one of the 64 BHT register write-enable inputs — specifically the one corresponding to the branch instruction that just executed.
3. The `branch_taken` signal (actual outcome from the ALU) determines whether the selected counter increments or decrements:
 - `branch_taken = 1` $\Rightarrow$ counter increments (saturates at 11)
 - `branch_taken = 0` $\Rightarrow$ counter decrements (saturates at 00)
4. The updated value is written into the selected register on the next rising clock edge.

3.6 Architecture Diagram

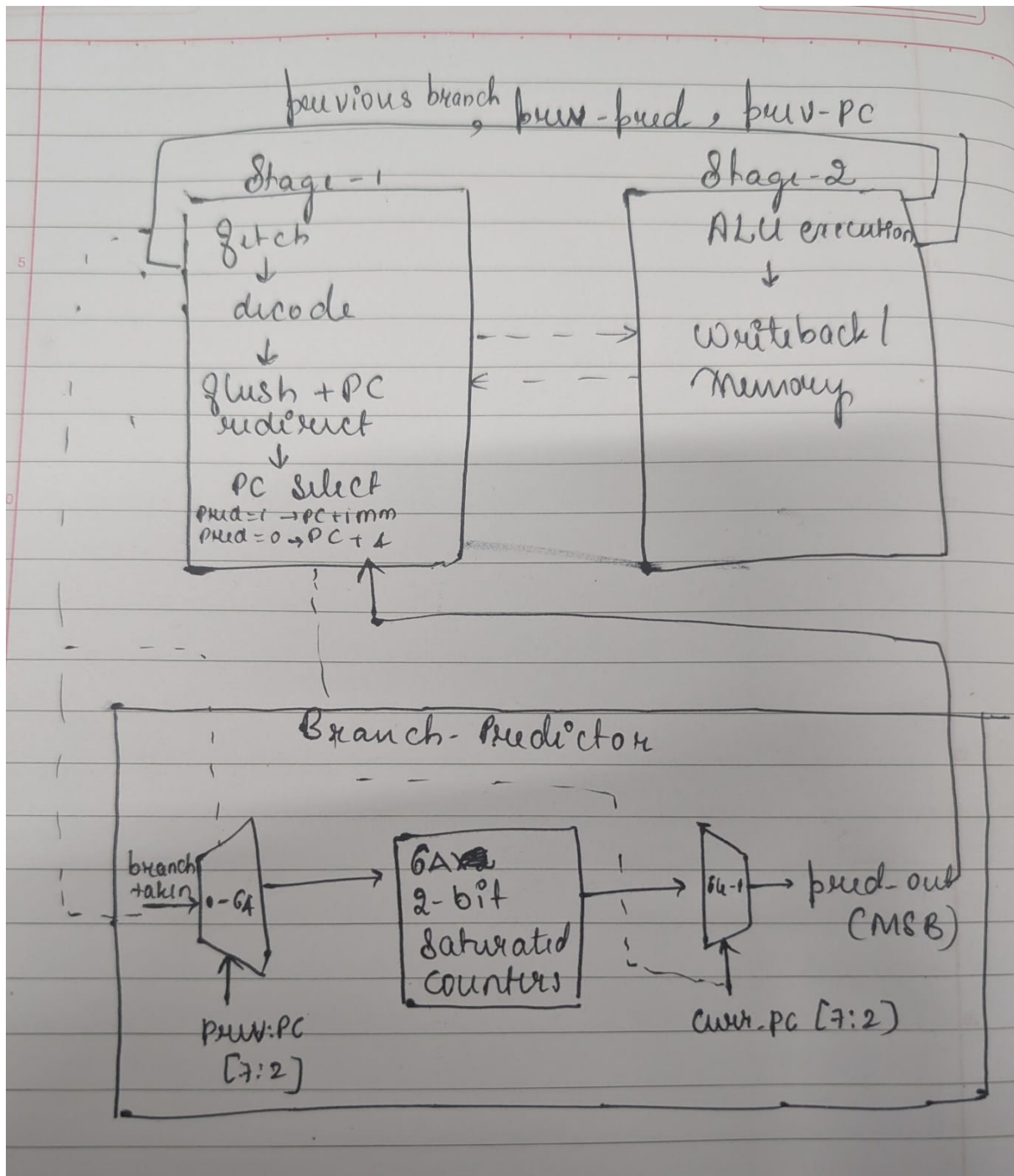


Figure 1: Branch Predictor Architecture

4 Integration into the Pipeline

4.1 Stage 1: Fetch, Decode and Prediction

The following sequence of operations happens in Stage 1 every cycle:

1. The instruction memory is read using the current PC, and the instruction is fetched.
2. The `idecoder` decodes the instruction and identifies whether it is a branch instruction (`branch_sel` signal goes high). It also extracts the B-type immediate `ImmB` and the `ready_stall` signal.
3. If it is a branch, the branch predictor is queried using `PC[7:2]` via the MUX read path, and `pred_out` is obtained.
4. The PC update MUX uses the prediction to choose the next PC:
 - `pred_out = 1`: next PC = PC + ImmB (jump to branch target)
 - `pred_out = 0`: next PC = PC + 4 (sequential fetch)
5. The prediction output `pred_out` is passed through the S1-S2 pipeline register along with the instruction and other decoded signals, so that Stage 2 can compare it against the actual outcome.

4.2 Stage 2: Execute and Branch Resolution

In Stage 2, the ALU executes the branch instruction and determines the actual outcome:

1. The ALU evaluates the branch condition using the register values. For example, for a BEQ instruction, it checks if `rs1 = rs2`. The result is a 1-bit signal: `alu_out` (1 = taken, 0 = not taken).
2. This actual outcome `alu_out` is compared with the prediction `pred_out` that was recorded in Stage 1.
3. The BHT is updated via the DEMUX write path using `prev_pc` and `branch_taken = alu_out`.
4. If `alu_out ≠ pred_out`, a **misprediction** has occurred and corrective action is taken (see Section 4.3).

4.3 Misprediction Handling

A misprediction means the instruction currently in Stage 1 was fetched from the wrong address. Since it has not yet executed, it must be **flushed** — its results must not be committed to any architectural state (no writeback, no memory operation). The pipeline then redirects the PC to the correct address.

Flushing the incorrect instruction:

The `ready_stall` signal is modified to suppress execution when a misprediction is detected. The updated logic is:

$$\text{ready_stall} = \text{ready_stall}_{\text{decoder}} \wedge \neg(\text{is_branch} \wedge \text{alu_out} \neq \text{pred_out})$$

When `ready_stall` is driven to 0 due to a misprediction, the Stage 2 datapath treats the instruction as a NOP (no operation) — the ALU does not commit its result, memory is not accessed, and no register is written back.

Redirecting the PC:

Since the wrong instruction was fetched based on the wrong prediction, the PC must be corrected. The PC redirect logic works as follows:

- If the pipeline predicted **not taken** (fetched PC + 4) but the branch was **actually taken**: the correct next PC is PC + ImmB. The PC MUX is overridden to select the branch target.
- If the pipeline predicted **taken** (fetched PC + ImmB) but the branch was **actually not taken**: the correct next PC is PC + 4. The PC MUX is overridden to select the sequential address.

This redirect happens in the same cycle that the misprediction is detected (end of Stage 2), so only **one bubble** (one wasted cycle) is introduced per misprediction.

4.4 Forwarding Logic

The data forwarding logic implemented in Lab 10 handles RAW (Read After Write) data hazards by forwarding register values from Stage 2 back to Stage 1. This mechanism is **completely unchanged** in this lab. Branch prediction is an orthogonal addition — it handles control hazards independently, and the two mechanisms do not interfere with each other.

5 Performance Counters

Four hardware performance counters are implemented in the top-level circuit using counters with clock-enable inputs. Each counter is driven by the main clock and is gated by a specific enable condition:

Counter	Enable Condition and Explanation
<code>instruction_count</code>	Enabled when <code>ready_stall = 1</code> . Since <code>ready_stall = 1</code> means the current instruction was not stalled and executed normally, this counts the number of instructions that were actually retired (committed).
<code>cycle_count</code>	Always enabled (no gate). Increments on every rising clock edge. This counts total wall-clock cycles including stalls and flushes, giving the total execution time in cycles.
<code>mispredictions</code>	Enabled when <code>is_branch = 1</code> AND <code>alu_out ≠ pred_out</code> . Counts every branch instruction whose actual outcome differed from the prediction.
<code>correctpredictions</code>	Enabled when <code>is_branch = 1</code> AND <code>alu_out = pred_out</code> . Counts every branch instruction whose actual outcome matched the prediction.

6 Results

The following table summarises the simulation results obtained by running the test programs from previous labs on the updated pipeline with the branch predictor enabled.

Test Program	Cycles	Instructions	Correct Pred.	Mispred.
mul	25f67	25da7	1447	1c0
no mul	948d3	907f1	a3c1	40e2

7 Summary

This lab successfully implements a complete dynamic branch predictor for a two-stage RISC-V pipeline. The key design decisions and contributions are:

- A **64-entry Branch History Table** was implemented in Logisim using 64 two-bit registers, indexed by `PC[7:2]`.
- Each entry uses a **2-bit saturating counter** with four states (Strongly Not Taken, Weakly Not Taken, Weakly Taken, Strongly Taken), providing hysteresis and better accuracy than a 1-bit scheme.

- The **read path** uses a 64-to-1 MUX: the current PC selects the counter, and the MSB gives the prediction in the same cycle as fetch.
- The **write path** uses a 1-to-64 DEMUX: `prev_pc` selects the BHT entry to update after Stage 2 resolves the branch.
- **Misprediction handling** flushes the wrong instruction by driving `ready_stall` to 0 and redirects the PC to the correct target, incurring only one penalty cycle per misprediction.
- The `ready_stall` signal was extended with a misprediction gate while all Lab 10 data forwarding logic was retained unchanged.
- **Four hardware counters** — cycles, instructions, correct predictions, and mispredictions — enable quantitative analysis of pipeline performance and predictor accuracy.